



University of Balamand

Faculty of Engineering

CPEN313 - Computer Embedded Systems Project

By

Riwa Karam

Fady Gebran

Mohamad Akoum

To

Dr. Rafic Ayoubi

May 15th, 2023

TABLE OF CONTENTS

TABLE OF CONTENTS	I
LIST OF TABLES.....	II
LIST OF FIGURES.....	III
OBJECTIVE	1
INTRODUCTION.....	2
DESCRIPTION	4
3.1 SCHEMATIC.....	4
3.2 DESCRIPTION.....	6
3.2.1 <i>“compare” – Internal Look</i>	6
3.2.2 <i>Inputs to the “compare” Blocks – External Look</i>	6
3.2.3 <i>Select Line “s1”</i>	8
3.2.4 <i>Select Line “s2”</i>	8
3.2.5 <i>Feedback to Registers</i>	9
3.2.6 <i>State Machine</i>	9
3.2.7 <i>“a_d” Select Line</i>	11
ALTERNATIVE SOLUTIONS.....	13
SIMULATION.....	14
CONCLUSION	16
APPENDIX A	17

LIST OF TABLES

TABLE 1: SORTING STEPS IN THE BITONIC SORTING (+ FOR ASCENDING AND – FOR DESCENDING)	2
TABLE 2: TRUTH TABLE - "COMPARATOR" BLOCK	6
TABLE 3: TRUTH TABLE – "IN1" AND "IN2" LOGIC IN "COMPARE" BLOCK.....	7
TABLE 4: MULTIPLEXERS - INPUTS TO THE DIFFERENT "COMPARE" BLOCKS.....	7
TABLE 5: TRUTH TABLE - RELATIONSHIP BETWEEN "S1" AND "S2"	8
TABLE 6: TRUTH TABLE – FEEDBACK VALUES LOGIC	9
TABLE 7: RELATIONSHIP BETWEEN COUNTER AND STATES.....	11
TABLE 8: "A_D" VALUES ACCORDING TO THE COUNTER VALUE	12

LIST OF FIGURES

FIGURE 1: SCHEMATIC - MAIN CIRCUIT PERFORMING THE BITONIC SORT	4
FIGURE 2: SCHEMATIC - "COMPARE" BLOCK	5
FIGURE 3: SCHEMATIC - DERIVATION OF "s2" FROM "s1"	5
FIGURE 4: STATE MACHINE	10
FIGURE 5: QUARTUS WAVEFORM SIMULATION - INPUT VALUES	14
FIGURE 6: QUARTUS WAVEFORM SIMULATION - OUTPUT VALUES	14

OBJECTIVE

The objective of this project is to present a detailed SystemVerilog implementation of the Bitonic Sorting Algorithm and demonstrate its functionality using Quartus software. The report aims to provide a comprehensive understanding of the algorithm's principles, explain the approach of the proposed solution, and guide the reader through the steps required to perform this algorithm in hardware and translate it to code. A schematic is needed to visually present the hardware design and its interconnections, as well as a simulation with the appropriate results.

INTRODUCTION

Sorting algorithms play a fundamental role in computer science and are utilized in a wide range of applications, from data processing to numerical analysis. The Bitonic Sorting Algorithm is a highly efficient parallel sorting algorithm that can effectively handle large datasets. By leveraging parallel processing architectures, the Bitonic Sorting Algorithm demonstrates excellent scalability and performance characteristics.

The algorithm's name is derived from the concept of a Bitonic sequence, which is a sequence that first monotonically increases and then monotonically decreases (or vice versa). The Bitonic Sorting Algorithm divides the input sequence into smaller subsequences, each of which is sorted in a Bitonic manner. These sorted subsequences are then recursively merged until the entire sequence is sorted in ascending order.

The Bitonic Sorting Algorithm has a time complexity of $O(\log^2 N)$, where N represents the number of elements in the input sequence. This complexity makes it highly efficient for large datasets and parallel processing architectures. Additionally, the algorithm lends itself well to hardware implementations due to its inherent parallelism.

In this report, we will delve into the implementation details of the Bitonic Sorting Algorithm, for a sequence of 16 numbers, composed of 8 bits each, using SystemVerilog, a hardware description language. We will provide a step-by-step guide on how to implement the algorithm, including the necessary modules, behavioral constructs, and submodules for merging the sorted subsequences. For the case we have, a sorting of 16 numbers, the following comparisons shown in the following table, in the order given, will be needed.

Table 1: Sorting Steps in the Bitonic Sorting (+ for Ascending and – for Descending)

0-1 (+)	2-3 (-)	4-5 (+)	6-7 (-)	8-9 (+)	10-11 (-)	12-13 (+)	14-15 (-)
0-2 (+)	1-3 (+)	4-6 (-)	5-7 (-)	8-10 (+)	9-11 (+)	12-14 (-)	13-15 (-)

0-1 (+)	2-3 (+)	4-5 (-)	6-7 (-)	8-9 (+)	10-11 (+)	12-13 (-)	14-15 (-)
0-4 (+)	1-5 (+)	2-6 (+)	3-7 (+)	8-12 (-)	9-13 (-)	10-14 (-)	11-15 (-)
0-2 (+)	1-3 (+)	4-6 (+)	5-7 (+)	8-10 (-)	9-11 (-)	12-14 (-)	13-15 (-)
0-1 (+)	2-3 (+)	4-5 (+)	6-7 (+)	8-9 (-)	10-11 (-)	12-13 (-)	14-15 (-)
0-8 (+)	1-9 (+)	2-10 (+)	3-11 (+)	4-12 (+)	5-13 (+)	6-14 (+)	7-15 (+)
0-4 (+)	1-5 (+)	2-6 (+)	3-7 (+)	8-12 (+)	9-13 (+)	10-14 (+)	11-15 (+)
0-2 (+)	1-3 (+)	4-6 (+)	5-7 (+)	8-10 (+)	9-11 (+)	12-14 (+)	13-15 (+)
0-1 (+)	2-3 (+)	4-5 (+)	6-7 (+)	8-9 (+)	10-11 (+)	12-13 (+)	14-15 (+)

Furthermore, this report aims to demonstrate the functionality of the Bitonic Sorting Algorithm by simulating the implementation using Quartus software. Quartus provides a comprehensive development environment for FPGA designs, allowing us to validate the correctness of the algorithm and assess its performance. By testing the implementation on Quartus, we can observe the sorting process, verify the output against expected results, and analyze the algorithm's efficiency in terms of sorting time.

Overall, this report serves as a practical and informative guide for understanding, implementing, and testing the Bitonic Sorting Algorithm in SystemVerilog using Quartus software. It provides a foundation for utilizing the algorithm in various hardware applications and facilitates a deeper comprehension of parallel sorting algorithms.

DESCRIPTION

3.1 Schematic

In this section, schematics of the hardware design and circuitry can be found. However, the logic and reasoning behind them can be found in the next section, **Description**. The following schematics were drawn on Logisim Evolution software. The purpose of showing the schematics is to show the output hardware of the code written for this project, in order to perform a Bitonic sort.

The following figure presents the main circuit of the algorithm in question; the registers to the left store the values of the 16 input numbers and the circuitry shows that there is feedback to those registers after performing the necessary comparisons and flipping/holding of the values.

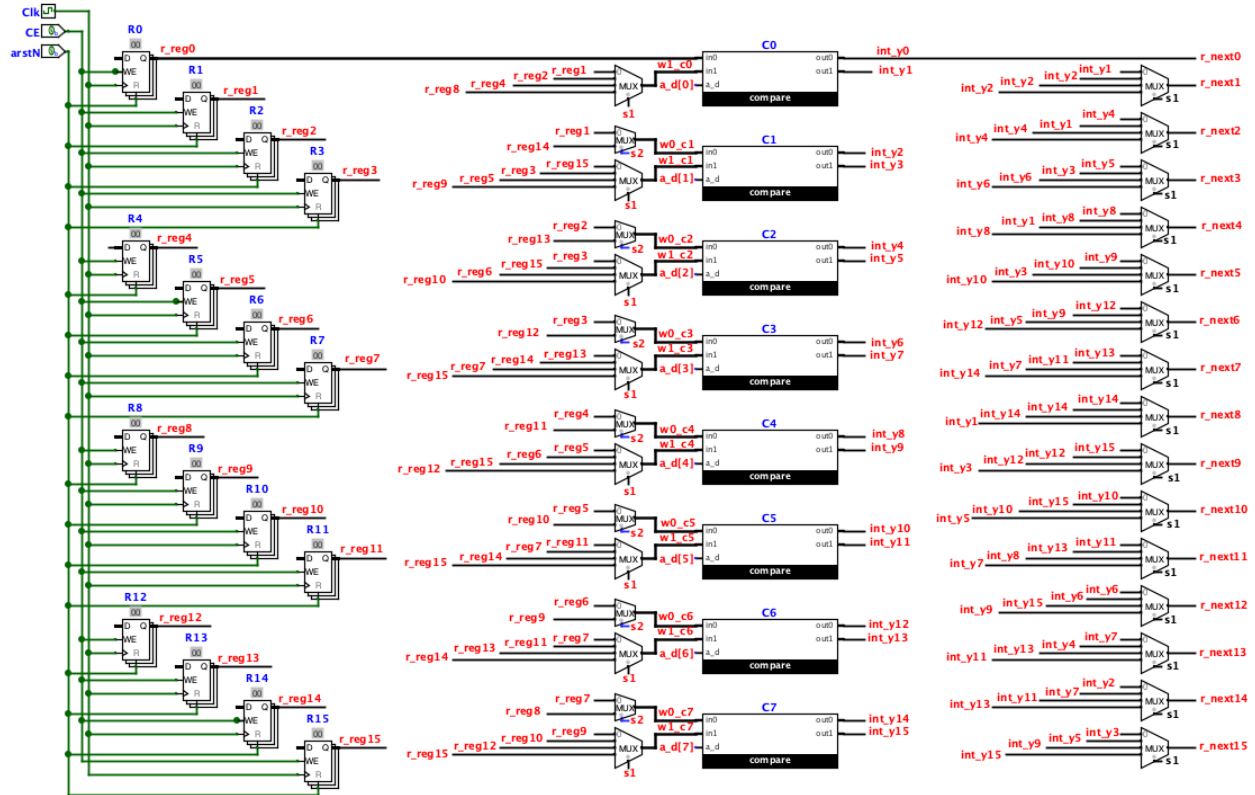


Figure 1: Schematic - Main Circuit Performing the Bitonic Sort

As show in **Figure 1**, there is a black box named “compare” used 8 times, having 3 inputs and 2 outputs. The following figure shows the circuit behind this block.

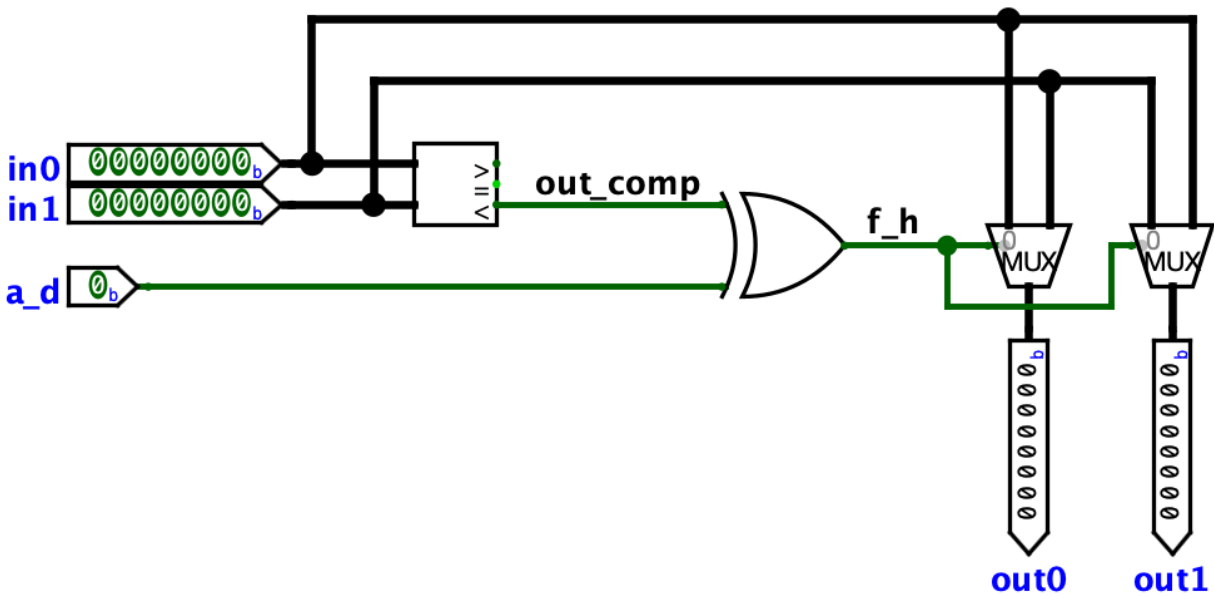


Figure 2: Schematic - "compare" Block

The select lines in the main circuit are represented as "s1" and "s2"; "s2" is derived from "s1" as it will be explained in a later stage in this report. The following figure shows the relationship between the two select lines.

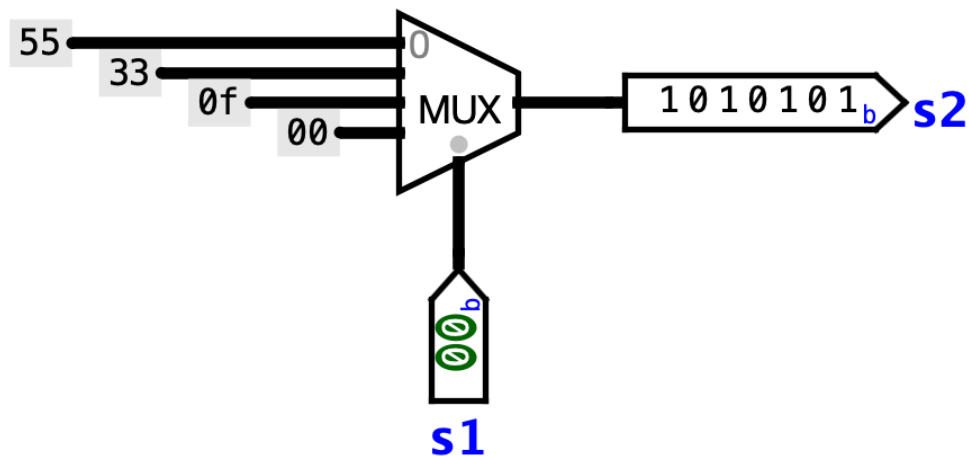


Figure 3: Schematic - Derivation of "s2" from "s1"

3.2 Description

In this section, each part of the hardware will be explained and the logic behind it will be presented, as well as the interconnections as show in the schematics above. The code can be found in **Appendix A**.

3.2.1 “compare” – Internal Look

The comparison part of the algorithm leading to the decision to either hold the values in place or flip them to the opposing position, is being done in the “compare” block of **Figure 2**. A “less than” comparator is used to compare the two input (8-bit) values; the resulting output will be a 1-bit value, i.e. either ‘0’ for greater or equal and ‘1’ for less than. This output is named “out_comp” in the code. The following truth table presents the workings of this block.

Table 2: Truth Table - “comparator” Block

a_d	out_comp	f_h	out0	out1
0	0	0	in0	in1
0	1	1	in1	in0
1	0	1	in1	in0
1	1	0	in0	in1

a_d: ascending (‘1’) or descending (‘0’)

f_h: flip (‘1’) or hold (‘0’)

3.2.2 Inputs to the “compare” Blocks – External Look

To comply with the workings of the given algorithm, at each step, the comparisons being made are between two numbers: from step to step, these numbers change. The following truth table shows the different needed comparisons to be made for the algorithm to be complete.

What can be noticed about this table is that where there is a drawn line between the lines with the ‘in1’ value being equal to 7 and 8, a figurative mirror can be seen, from which we can

take advantage of in the hardware. Each pair of lines (i.e. 7 and 8, 6 and 9, 5 and 10, etc.) complement each other in terms of empty places (X) and filled ones. That is why they can be merged under one multiplexer with the corresponding in1 values in another. The combination of the two multiplexers forms the two values to compare.

Table 3: Truth Table – “in1” and “in2” Logic in "compare" block

in0	in1			
0	1	2	4	8
1	X	3	5	9
2	3	X	6	10
3	X	X	7	11
4	5	6	X	12
5	X	7	X	13
6	7	X	X	14
7	X	X	X	15
8	9	10	11	X
9	X	11	12	X
10	11	X	13	X
11	X	X	15	X
12	13	14	X	X
13	X	15	X	X
14	15	X	X	X
15	X	X	X	X

After combining the corresponding lines, the resulting multiplexers will look as is seen in the following table.

Table 4: Multiplexers - Inputs to the Different "compare" Blocks

in0		in1			
0	0	1	2	4	8
1	14	15	3	5	9

2	13	3	15	6	10
3	12	13	14	7	11
4	11	5	6	15	12
5	10	11	7	14	13
6	9	7	11	13	14
7	8	9	10	11	15

After choosing the inputs to each multiplexer, select lines are needed; one for the multiplexer representing “in0”, which will be a 1-bit select line named “s2” in the code, and another for the multiplexer representing “in1”, which will be a 2-bits select line named “s1” in the code.

3.2.3 Select Line “s1”

In this algorithm, 4 stages can be found: compare consecutive values, values spaced with 1 index in between, values spaced with 2 indexes in between, and values spaced with 8 indexes in between. The select line “s1” will represent those 4 stages (00, 01, 10, 11), which will in turn represent the 4 states, which will be explained in a later stage.

3.2.4 Select Line “s2”

To match the needed values in both multiplexers, the select line “s2” must satisfy certain values depending on the stage. The following table represents the relationship between “s1” and “s2”.

Table 5: Truth Table - Relationship Between “s1” and “s2”

S1	S2
00	0x55
01	0x33
10	0xF0
11	0x00

3.2.5 Feedback to Registers

In order to be able to put values back into the register after each step, there is a need to get the corresponding output to the corresponding input. The following table shows which internal output should get fed back into the registers, given the displacement in the comparisons and complex choice of value comparisons.

Table 6: Truth Table – Feedback Values Logic

R	00	01	10	11
0	int_y0	int_y0	int_y0	int_y0
1	int_y1	int_y2	int_y2	int_y2
2	int_y4	int_y1	int_y4	int_y4
3	int_y5	int_y3	int_y6	int_y6
4	int_y8	int_y8	int_y1	int_y8
5	int_y9	int_y10	int_y3	int_y10
6	int_y12	int_y9	int_y5	int_y12
7	int_y13	int_y11	int_y7	int_y14
8	int_y14	int_y14	int_y14	int_y1
9	int_y15	int_y12	int_y12	int_y3
10	int_y10	int_y15	int_y10	int_y5
11	int_y11	int_y13	int_y8	int_y7
12	int_y6	int_y6	int_y15	int_y9
13	int_y7	int_y4	int_y13	int_y11
14	int_y2	int_y7	int_y11	int_y13
15	int_y3	int_y5	int_y9	int_y15

3.2.6 State Machine

In order to control the flow of the comparison process, a control unit should be implemented, indicating the values of needed select lines and control lines. The control unit is composed of a state machine with 4 states; the following figure presents this FSM.

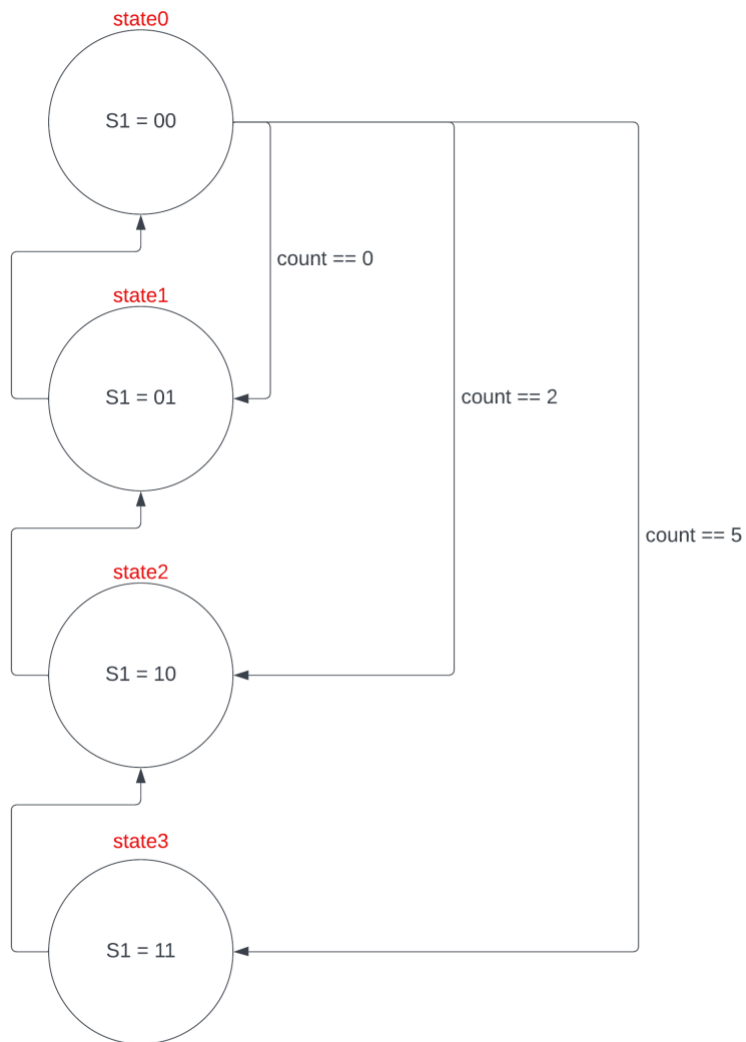


Figure 4: State Machine

The only affected select line, as it can be seen in the above figure is “s1”. However, a kind of domino effect happens when “s1” changes values; the select line “s1” affects the select line “s2”, the multiplexers leading to the inputs of the comparators and the ones leading to the feedback into the registers.

As for the count, **Table 1** shows us that there are 10 steps needed to get to the final output, given that we have 16 numbers in the sequence to be sorted using the Bitonic sort algorithm. Since

we have 4 states, with some states getting repeated throughout the 10 steps, a counter can help indicate to which state to go to. At each clock rising edge, the counter is incremented by 1 until it reaches the value of 10. With this counter a good management of the states can be achieved.

The following table is presented to further explain the workings of the state machine and its relationship to the counter.

Table 7: Relationship Between Counter and States

Counter	State
0	state0
1	state1
2	state0
3	state2
4	state1
5	state0
6	state3
7	state2
8	state1
9	state0

3.2.7 “a_d” Select Line

Following the counter and the explanation in the FSM, the ascending or descending choice needs to be explained. As seen in the code, the “a_d” bit of the comparators is determined by the counter value. The following table shows the corresponding values according to the counter value.

Table 8: "a_d" Values According to the Counter Value

Counter	"a_d"
0	10011001
1	11000011
2	10100101
3	11110000
4	11001100
5	10101010
6	11111111
7	11111111
8	11111111
9	11111111

ALTERNATIVE SOLUTIONS

There are many ways to implement this project using SystemVerilog and hardware implementations. The approach we chose is of a global state machine controlling the main signal as well as a custom comparator block.

A first alternative approach would be to use two state machines: a global one and a local one. The global FSM would resemble the one we chose to implement. As for the local one, it would control the comparison step instead of a logic submodule. It would consist of having to choose whether to flip or hold the input values; so, it would have the same concept as the “compare” block, but it would be implemented using states. For our project, we did not use this approach since we found it more straightforward to implement a logic submodule, having the same efficiency in hardware and same speed.

Another alternative method would be to control all signals inside the states of the global state machine, independent of a counter. Thus, the FSM would be composed of 10 states instead of 4 and in each one all select lines and needed values would be determined. This approach is less efficient and code friendly than the one we used that is why we stucked to the one we explained above.

SIMULATION

After designing the hardware and writing the code, we compiled the code on a new Quartus project after which we tested and simulated an example of a sequence to be sorted. The following figures show the simulation output.

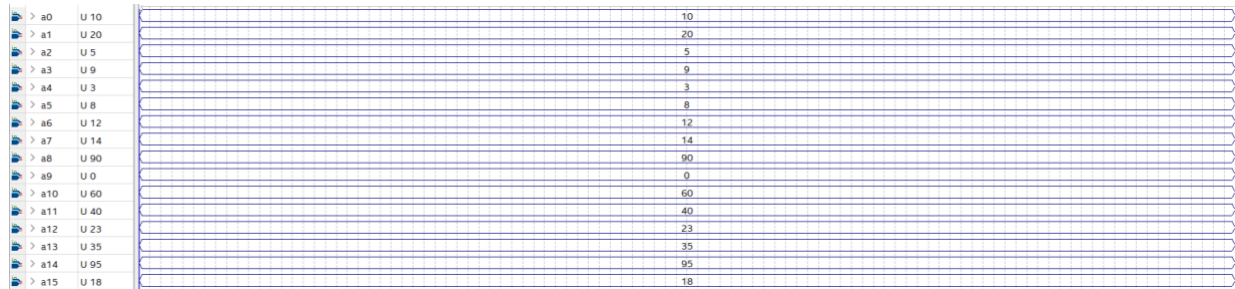


Figure 5: Quartus Waveform Simulation - Input Values

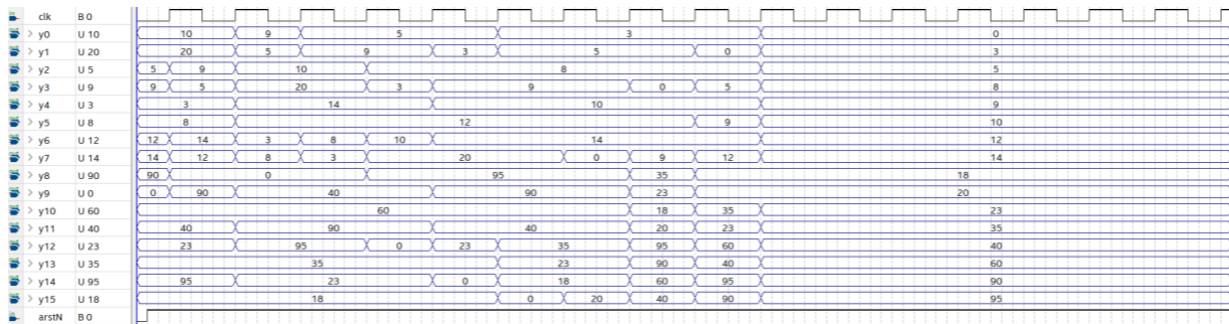


Figure 6: Quartus Waveform Simulation - Output Values

At first, the “arstN” is set to ‘0’ to initialize all variables; after that, it is always set to ‘1’. At each rising edge, the code jumps from state to state to complete all 10 steps of the sorting algorithm in question. The values are either flipped or held in place at each step; this can be clearly seen in **Figure 6**. The final output value, as seen to the right of **Figure 6** is held due to CE being set to ‘0’ after the 10 steps of the code.

As shown in this simulation example, the logic of the hardware and the code written for this project lead to a final sorted in ascending order of a sequence containing 16 numbers of 8 bits each.

We tested the code for several clock values and even at extremely fast clocks (0.01ns), the sequence will get sorted properly. However, for visibility purposes, the above simulation is set with a clock of period 30ns.

CONCLUSION

In conclusion, our project presented a SystemVerilog implementation of the Bitonic Sorting Algorithm. The solution involved creating a module to encapsulate the algorithm, implementing the sorting function and merging submodules, and simulating the code using Quartus software. The results obtained from the simulation provided insights into the correctness and performance of the algorithm. The implemented Bitonic Sorting Algorithm can be further optimized and extended for practical applications in parallel processing architectures.

APPENDIX A

The following code is in the top-level SystemVerilog file compiled in Quartus.

```
module bitonic_sort (input logic [7:0] a0, a1, a2, a3, a4, a5, a6, a7, a8,
                    a9, a10, a11, a12, a13, a14, a15,
                    input logic clk, arstN,
                    output logic [7:0] y0, y1, y2, y3, y4, y5, y6, y7,
                    y8, y9, y10, y11, y12, y13, y14, y15);

typedef enum logic [3:0] {state0 = 4'b0001,
                        state1 = 4'b0010,
                        state2 = 4'b0100,
                        state3 = 4'b1000} state_t;
state_t state_next, state_reg;

logic [7:0] r_next0, r_next1, r_next2, r_next3,
            r_next4, r_next5, r_next6, r_next7,
            r_next8, r_next9, r_next10, r_next11,
            r_next12, r_next13, r_next14, r_next15,
            r_reg0, r_reg1, r_reg2, r_reg3,
            r_reg4, r_reg5, r_reg6, r_reg7,
            r_reg8, r_reg9, r_reg10, r_reg11,
            r_reg12, r_reg13, r_reg14, r_reg15;

logic CE;

logic [7:0] w0_c0, w0_c1, w0_c2, w0_c3, w0_c4, w0_c5, w0_c6, w0_c7,
            w1_c0, w1_c1, w1_c2, w1_c3, w1_c4, w1_c5, w1_c6, w1_c7;
logic [7:0] a_d;
logic [1:0] s1;
logic [6:0] s2;
logic [7:0] int_y0, int_y1, int_y2, int_y3, int_y4, int_y5, int_y6, int_y7,
            int_y8, int_y9, int_y10, int_y11, int_y12, int_y13, int_y14,
int_y15;

logic [3:0] count_reg, count_next;

always_ff @(posedge clk, negedge arstN)
    if(!arstN) begin
        state_reg <= state0;
        r_reg0 <= a0;
        r_reg1 <= a1;
        r_reg2 <= a2;
        r_reg3 <= a3;
        r_reg4 <= a4;
        r_reg5 <= a5;
        r_reg6 <= a6;
        r_reg7 <= a7;
        r_reg8 <= a8;
        r_reg9 <= a9;
        r_reg10 <= a10;
        r_reg11 <= a11;
        r_reg12 <= a12;
        r_reg13 <= a13;
        r_reg14 <= a14;
```

```

        r_reg15 <= a15;
        count_reg <= 'b0;
    end
    else if(CE) begin
        state_reg <= state_next;
        r_reg0 <= r_next0;
        r_reg1 <= r_next1;
        r_reg2 <= r_next2;
        r_reg3 <= r_next3;
        r_reg4 <= r_next4;
        r_reg5 <= r_next5;
        r_reg6 <= r_next6;
        r_reg7 <= r_next7;
        r_reg8 <= r_next8;
        r_reg9 <= r_next9;
        r_reg10 <= r_next10;
        r_reg11 <= r_next11;
        r_reg12 <= r_next12;
        r_reg13 <= r_next13;
        r_reg14 <= r_next14;
        r_reg15 <= r_next15;
        count_reg <= count_next;
    end

assign y0 = r_reg0;
assign y1 = r_reg1;
assign y2 = r_reg2;
assign y3 = r_reg3;
assign y4 = r_reg4;
assign y5 = r_reg5;
assign y6 = r_reg6;
assign y7 = r_reg7;
assign y8 = r_reg8;
assign y9 = r_reg9;
assign y10 = r_reg10;
assign y11 = r_reg11;
assign y12 = r_reg12;
assign y13 = r_reg13;
assign y14 = r_reg14;
assign y15 = r_reg15;

always_comb begin
    case (s1)
        2'b00: begin
            s2 = 7'b1010101;
            w1_c0 = r_reg1;
            w1_c1 = r_reg15;
            w1_c2 = r_reg3;
            w1_c3 = r_reg13;
            w1_c4 = r_reg5;
            w1_c5 = r_reg11;
            w1_c6 = r_reg7;
            w1_c7 = r_reg9;
        end
        2'b01: begin
            s2 = 7'b0110011;
            w1_c0 = r_reg2;

```

```

        w1_c1 = r_reg3;
        w1_c2 = r_reg15;
        w1_c3 = r_reg14;
        w1_c4 = r_reg6;
        w1_c5 = r_reg7;
        w1_c6 = r_reg11;
        w1_c7 = r_reg10;
    end
    2'b10: begin
        s2 = 7'b0001111;
        w1_c0 = r_reg4;
        w1_c1 = r_reg5;
        w1_c2 = r_reg6;
        w1_c3 = r_reg7;
        w1_c4 = r_reg15;
        w1_c5 = r_reg14;
        w1_c6 = r_reg13;
        w1_c7 = r_reg12;
    end
    2'b11: begin
        s2 = 7'b0000000;
        w1_c0 = r_reg8;
        w1_c1 = r_reg9;
        w1_c2 = r_reg10;
        w1_c3 = r_reg11;
        w1_c4 = r_reg12;
        w1_c5 = r_reg13;
        w1_c6 = r_reg14;
        w1_c7 = r_reg15;
    end
endcase
end

assign w0_c1 = s2[6] ? r_reg14 : r_reg1;
assign w0_c2 = s2[5] ? r_reg13 : r_reg2;
assign w0_c3 = s2[4] ? r_reg12 : r_reg3;
assign w0_c4 = s2[3] ? r_reg11 : r_reg4;
assign w0_c5 = s2[2] ? r_reg10 : r_reg5;
assign w0_c6 = s2[1] ? r_reg9 : r_reg6;
assign w0_c7 = s2[0] ? r_reg8 : r_reg7;

always_comb begin
    a_d = 'b0;
    CE = 1'b1;
    unique case (count_reg)
        4'b0000: a_d = 8'b10011001;
        4'b0001: a_d = 8'b11000011;
        4'b0010: a_d = 8'b10100101;
        4'b0011: a_d = 8'b11110000;
        4'b0100: a_d = 8'b11001100;
        4'b0101: a_d = 8'b10101010;
        4'b0110: a_d = 8'b11111111;
        4'b0111: a_d = 8'b11111111;
        4'b1000: a_d = 8'b11111111;
        4'b1001: a_d = 8'b11111111;
        4'b1010: CE = 1'b0;
    endcase
end

```

```

end

compare c0(r_reg0, w1_c0, a_d[7], int_y0, int_y1);
compare c1(w0_c1, w1_c1, a_d[6], int_y2, int_y3);
compare c2(w0_c2, w1_c2, a_d[5], int_y4, int_y5);
compare c3(w0_c3, w1_c3, a_d[4], int_y6, int_y7);
compare c4(w0_c4, w1_c4, a_d[3], int_y8, int_y9);
compare c5(w0_c5, w1_c5, a_d[2], int_y10, int_y11);
compare c6(w0_c6, w1_c6, a_d[1], int_y12, int_y13);
compare c7(w0_c7, w1_c7, a_d[0], int_y14, int_y15);

always_comb begin
    r_next0 = int_y0;
    case (s1)
        2'b00: begin
            r_next1 = int_y1;
            r_next2 = int_y4;
            r_next3 = int_y5;
            r_next4 = int_y8;
            r_next5 = int_y9;
            r_next6 = int_y12;
            r_next7 = int_y13;
            r_next8 = int_y14;
            r_next9 = int_y15;
            r_next10 = int_y10;
            r_next11 = int_y11;
            r_next12 = int_y6;
            r_next13 = int_y7;
            r_next14 = int_y2;
            r_next15 = int_y3;
        end
        2'b01: begin
            r_next1 = int_y2;
            r_next2 = int_y1;
            r_next3 = int_y3;
            r_next4 = int_y8;
            r_next5 = int_y10;
            r_next6 = int_y9;
            r_next7 = int_y11;
            r_next8 = int_y14;
            r_next9 = int_y12;
            r_next10 = int_y15;
            r_next11 = int_y13;
            r_next12 = int_y6;
            r_next13 = int_y4;
            r_next14 = int_y7;
            r_next15 = int_y5;
        end
        2'b10: begin
            r_next1 = int_y2;
            r_next2 = int_y4;
            r_next3 = int_y6;
            r_next4 = int_y1;
            r_next5 = int_y3;
            r_next6 = int_y5;
            r_next7 = int_y7;
            r_next8 = int_y14;
        end
    end
end

```

```

        r_next9 = int_y12;
        r_next10 = int_y10;
        r_next11 = int_y8;
        r_next12 = int_y15;
        r_next13 = int_y13;
        r_next14 = int_y11;
        r_next15 = int_y9;
    end
    2'b11: begin
        r_next1 = int_y2;
        r_next2 = int_y4;
        r_next3 = int_y6;
        r_next4 = int_y8;
        r_next5 = int_y10;
        r_next6 = int_y12;
        r_next7 = int_y14;
        r_next8 = int_y1;
        r_next9 = int_y3;
        r_next10 = int_y5;
        r_next11 = int_y7;
        r_next12 = int_y9;
        r_next13 = int_y11;
        r_next14 = int_y13;
        r_next15 = int_y15;
    end
endcase
end

always_comb begin
    unique case (state_reg)
        state0: begin
            s1 = 2'b00;
            state_next = (count_reg == 4'b0000) ? state1 :
                (count_reg == 4'b0010) ? state2 :
                state3;
        end
        state1: begin
            s1 = 2'b01;
            state_next = state0;
        end
        state2: begin
            s1 = 2'b10;
            state_next = state1;
        end
        state3: begin
            s1 = 2'b11;
            state_next = state2;
        end
    endcase
end

assign count_next = count_reg + 4'b0001;

endmodule

```

The following code is for the “comparator” SystemVerilog file compiled in Quartus; it is used in the top-level code seen above.

```
module compare (input logic [7:0] in0, in1,
                input logic a_d,
                output logic [7:0] out0, out1);

    logic out_comp, f_h;
    assign out_comp = in0 < in1;
    assign f_h = a_d ^ out_comp;
    assign out0 = f_h ? in1 : in0;
    assign out1 = f_h ? in0 : in1;

endmodule
```